

Cosmic Briefing

Tarea 9. Workflow inteligente con n8n, Ollama y Groq

Programación de Inteligencia Artificial
CE en Inteligencia Artificial y Big Data

Antonio Ferrer Martínez

1.º BDAT

Abril 2026

Vídeo demo: <https://www.youtube.com/watch?v=JRhiNauBYrM>

Índice de contenidos

Índice de contenidos	2
Glosario rápido.....	iError! Marcador no definido.
1. Objetivo del workflow	5
1.1 Objetivos específicos del proyecto	6
2. Arquitectura: qué nodos uso y por qué	8
2.1 Visión general en tres ramas	8
2.2 Tecnologías que enchufa el workflow	9
2.2.1 n8n.....	9
2.2.2 Ollama y los modelos descargados.....	9
2.2.3 Groq	9
2.2.4 Telegram Bot API.....	10
2.2.5 Google Apps Script	10
2.3 Las 8 fuentes de datos externas.....	10
2.3.1 NASA APOD	10
2.3.2 Where the ISS at.....	11
2.3.3 NASA NeoWs.....	11
2.3.4 Launch Library 2 (TheSpaceDevs).....	11
2.3.5 Open-Meteo	11
2.3.6 Sunrise-Sunset.org	12
2.3.7 arXiv astro-ph (RSS).....	12
2.3.8 SpaceNews (RSS).....	12
2.4 Rama A — Briefing diario nodo a nodo	12
2.4.1 Nodo 1 — Schedule 08:00 diario.....	12
2.4.2 Nodo 2 — Configuración (Code).....	12
2.4.3 Nodos 3 a 10 — 8 nodos HTTP a las fuentes externas.....	13
2.4.4 Nodo 11 — Consolidar datos (Code).....	13
2.4.5 Nodo 12 — Embeddings Ollama	14
2.4.6 Nodo 13 — Clustering y prompt (Code)	14
2.4.7 Nodo 14 — LLM Ollama (qwen2.5:1.5b)	15
2.4.8 Nodo 15 — Telegram (briefing)	16
2.4.9 Nodo 16 — Google Sheets (log).....	16
2.5 Rama B — Chatbot nodo a nodo.....	16
2.5.1 Nodo 17 — Telegram Trigger (chatbot).....	16
2.5.2 AI Agent y sub-nodos.....	16

2.5.3 Nodo 18 — Telegram (respuesta chatbot)	17
2.6 Rama C — Manejo de errores.....	17
2.7 Apps Script (Code.gs)	17
2.7.1 doPost: cómo se guarda un briefing	18
2.7.2 doGet: tres acciones en un solo endpoint.....	18
2.7.3 renderDashboard: el panel HTML.....	18
2.8 Decisiones técnicas y trazabilidad de ideas	19
2.8.1 Origen del proyecto	19
2.8.2 Local en lugar del n8n del profesor	20
2.8.3 Groq para el chatbot, Ollama para el briefing	20
2.8.4 Un solo workflow	20
2.8.5 onError: continueRegularOutput en las 8 fuentes.....	20
3. Instrucciones de configuración.....	22
3.1 Requisitos previos	22
3.2 Arranque del kit en Docker	22
3.3 Descarga de modelos en Ollama	22
3.4 Túnel ngrok	23
3.5 Credenciales en n8n	23
3.6 Despliegue del Apps Script	23
3.7 Importación y activación del workflow.....	24
4. Capturas de pantalla	25
4.1 Vistas del workflow.....	25
4.2 Ejecución exitosa	27
4.3 Mensajes en Telegram	27
4.4 Persistencia y panel.....	29
4.5 ngrok funcionando.....	30
5. Problemas encontrados y cómo los resolví	31
6. Pruebas funcionales	33
7. Conclusiones y trabajos futuros	34
7.1 Conclusiones	34
7.2 Trabajos futuros.....	34
8. Recursos y referencias.....	36
8.1 Plataforma y entorno	36
8.2 Modelos y LLM	36
8.3 APIs de datos del briefing.....	36

8.4 Telegram y Apps Script	37
8.5 Inspiración para el formato del proyecto	37

1. Objetivo del workflow

Esta tarea plantea construir un workflow inteligente en n8n que integre componentes de Inteligencia Artificial y resuelva un problema realista. El enunciado pide un nodo LLM, una fuente de datos externa, una salida útil, al menos cinco nodos sin contar el trigger y un mecanismo básico de manejo de errores. Avisa expresamente de que no vale presentar un RAG simple (es el ejemplo de clase) y propone varias ideas como punto de partida.

Mi proyecto se llama Cosmic Briefing. Cada mañana a las 08:00 manda al móvil por Telegram un boletín de astronomía y espacio armado con ocho fuentes públicas en directo: NASA APOD para la imagen del día, Where-ISS para la posición de la Estación Espacial, NASA NeoWs para los asteroides cercanos, Launch Library 2 para los próximos lanzamientos, Open-Meteo y Sunrise-Sunset para las condiciones del cielo esta noche, arXiv para los papers de astrofísica del día y SpaceNews para las noticias periódicas.

Esos titulares no se mandan tal cual al LLM. Pasan por un modelo de embeddings (``nomic-embed-text``) que devuelve vectores de 768 dimensiones. Después se agrupan con un clustering greedy por similitud coseno con umbral 0,70. Los grupos resultantes se inyectan en el prompt junto con los datos crudos, y un LLM local de Ollama (``qwen2.5:1.5b``) escribe el briefing en castellano. El mensaje llega a Telegram con la imagen del APOD incrustada y el enlace a un panel HTML público con el histórico. Cada briefing también se persiste en Google Sheets vía un Apps Script propio.

A la rama del briefing diario le añadí una segunda rama: un chatbot que escucha al mismo bot de Telegram. Cuando le mandas un mensaje, un AI Agent (Groq, ``llama-3.3-70b-versatile``) decide si es small talk o si tiene que invocar alguna de las dos herramientas que tiene a mano:

``ultimos_briefings`` y ``buscar_briefing``. Las dos tiran del Apps Script

para consultar la hoja de Sheets. El agente tiene memoria conversacional por chat, así que recuerda lo que le has preguntado antes en el mismo hilo.

Y una tercera rama, la C, que es un Error Trigger global: si cualquier nodo del workflow falla, me llega un mensaje de aviso a Telegram con el nombre del nodo y el error. Las tres ramas viven en el mismo fichero JSON, comparten bot, hoja y modelo de embeddings.

El proyecto NO es un RAG. No hay vector store ni indexación de documentos. Lo que hago es coger ocho fuentes en directo, agruparlas por tema con embeddings y narrar el conjunto con un LLM. La sección 5 del enunciado proponía como ejemplo un «resumidor de noticias» con cron + RSS + LLM + Telegram, y esa fue la chispa de la idea, pero el proyecto la extiende bastante.

1.1 Objetivos específicos del proyecto

- Que un workflow se ejecute solo cada día a las 08:00 sin intervención manual.
- Combinar ocho fuentes científicas reales en un único mensaje legible y compacto.
- Aplicar IA de verdad y no de adorno: embeddings semánticos para detectar temas comunes y un LLM local para escribir el briefing.
- Construir un chatbot del mismo bot de Telegram con AI Agent, memoria conversacional y herramientas reales que consulten el histórico de briefings.
- Usar el menor número de servicios de pago posible: todo el briefing diario corre en local con Ollama, y el chatbot va por Groq que es gratuito.
- Tolerar fallos parciales: si una API se cae, el resto del briefing se entrega igual con los datos disponibles.

- Mantener todo en un único fichero
`apellido\_nombre\_workflow.json` como pide el enunciado, sin sub-workflows ni infraestructura accesoria.
- Documentar el proceso (este documento) y los problemas encontrados, para que otra persona pueda reproducir el proyecto a partir de aquí.

2. Arquitectura: qué nodos uso y por qué

2.1 Visión general en tres ramas

El sistema vive en un único workflow de n8n con tres ramas independientes que no se cruzan. Cada una tiene su propio trigger:

Rama	Trigger	Lo que hace
A — Briefing diario	Schedule cron 0 8 * * *	8 APIs → consolidar → embeddings → clustering → LLM Ollama → Telegram + Sheets
B — Chatbot	Telegram Trigger	AI Agent (Groq) con memoria + 2 tools que consultan Sheets → respuesta a Telegram
C — Errores	Error Trigger	Si cualquier nodo del workflow falla, aviso por Telegram al admin

Las tres ramas comparten bot, hoja y modelo de embeddings. El briefing diario escribe en Sheets; el chatbot lee de Sheets a través de las herramientas. La rama de errores notifica fallos del propio workflow.

Todo esto corre encima del `self-hosted-ai-starter-kit`, un docker-compose oficial mantenido por el equipo de n8n que despliega cuatro servicios: n8n (motor de workflows), Ollama (servidor local de modelos), Postgres (base de datos interna que usa el propio n8n) y Qdrant (base vectorial que el kit incluye por defecto y que en este proyecto NO uso, ya que no hago RAG; el contenedor lo dejo parado para no malgastar memoria).

La instancia de n8n se expone al exterior con ngrok, usando un dominio estático del plan gratuito (`tapping-widget-entering.ngrok-free.dev`), para que Telegram pueda entregar los mensajes del chatbot al webhook local.

2.2 Tecnologías que enchufa el workflow

2.2.1 n8n

n8n es una plataforma de automatización tipo low-code en la que dibujas el flujo nodo a nodo. Cada nodo encapsula una operación: un trigger temporal, una llamada HTTP, ejecutar JavaScript, un nodo de IA, etc. Los datos fluyen entre nodos como objetos JSON, y puedes inspeccionar lo que entra y lo que sale en cada paso. La instancia que uso es local, dentro de un Docker, con la última versión LTS. Accedo por el puerto 5678 en el navegador.

2.2.2 Ollama y los modelos descargados

Ollama es un servidor de modelos LLM y embeddings que se ejecuta en local. Una vez instalado (en este caso, dentro del contenedor del kit), descargo modelos open-source con ``ollama pull <nombre>`` y los sirvo por su API HTTP en el puerto 11434.

- ``qwen2.5:1.5b`` — modelo de generación de texto de 1.5B de parámetros. Lo uso para escribir el briefing en castellano. Lo elegí porque encaja en CPU sin tener que cargar gigas de VRAM y narra bien en español sin alucinar demasiado.
- ``nomic-embed-text`` — modelo de embeddings con dimensión 768. Lo uso para vectorizar los titulares antes de hacer el clustering. Pesa unos 270 MB y es rápido en CPU.

Probé ``qwen2.5:3b`` para el chatbot pensando en function calling, pero en CPU tarda entre 30 y 60 segundos por respuesta y eso rompe el webhook de Telegram. Por eso al final el chatbot lo movile a Groq.

2.2.3 Groq

Groq es un proveedor de inferencia LLM en la nube famoso por su velocidad: usa hardware especializado (LPUs) y responde en milisegundos. Tiene un tier gratuito de 14400 peticiones al día con ``llama-3.3-70b-versatile``, que es el modelo que uso para el chatbot. La API key se saca de ``https://console.groq.com/keys``. En n8n se configura

como credencial de tipo «Groq» y queda disponible para los nodos langchain.

2.2.4 Telegram Bot API

El bot de Telegram lo creé con BotFather. Me da un token tipo ``8642010850:AAH_...`` que es lo que usan tanto la rama del briefing (para enviar mensajes con ``sendMessage``) como el Telegram Trigger del chatbot (para recibir webhooks). Para que Telegram pueda entregar webhooks a una n8n local, necesito una URL pública. Eso lo resuelve ngrok.

2.2.5 Google Apps Script

Apps Script es el motor de scripting de Google Workspace. En este proyecto lo uso para añadir cuatro endpoints sobre la hoja Cosmic Briefing:

- ``POST`` — añade una fila al final de la hoja con los datos del briefing.
- ``GET ?action=last&n=N`` — devuelve las últimas N filas en JSON, para la tool ``ultimos_briefings``.
- ``GET ?action=search&q=X`` — devuelve las filas que contienen ``X`` en briefing/papers/noticias/título APOD, para la tool ``buscar_briefing``.
- ``GET ?action=dashboard`` — devuelve un panel HTML con tarjetas, una por briefing.

2.3 Las 8 fuentes de datos externas

2.3.1 NASA APOD

Imagen astronómica del día. Devuelve título, fecha, URL de la imagen y explicación. La API necesita una API key gratuita que se saca en ``https://api.nasa.gov/``.

```
https://api.nasa.gov/planetary/apod?api_key=<KEY>
```

De la respuesta cojo `title`, `date`, `url` y los primeros 400 caracteres de `explanation`. La URL es la que pego en el mensaje de Telegram, y Telegram la incrusta como imagen automáticamente.

2.3.2 Where the ISS at

Devuelve la posición de la Estación Espacial Internacional en tiempo real. Es gratuita y no necesita clave.

```
https://api.wheretheiss.at/v1/satellites/25544
```

Con la lat/lon de la ISS y la lat/lon de Cartagena calculo la distancia entre ambas con la fórmula del haversine. Si la ISS está a menos de 2000 km marco una bandera `sobre\_zona = true`, que después aparece como etiqueta amarilla en el panel HTML.

2.3.3 NASA NeoWs

Asteroides que se acercan a la Tierra hoy. Misma API key que APOD. La URL la parametrizo con la fecha de hoy:

```
https://api.nasa.gov/neo/rest/v1/feed?start_date=YYYY-MM-DD&end_date=YYYY-MM-DD&api_key=<KEY>
```

De cada asteroide saco nombre, diámetro medio, distancia en distancias lunares (más fácil de visualizar que en km) y la bandera `is\_potentially\_hazardous\_asteroid`. Con tres me basta para el briefing.

2.3.4 Launch Library 2 (TheSpaceDevs)

Lista los próximos lanzamientos espaciales. Es gratuita y no hace falta clave.

```
https://ll.thespacedevs.com/2.2.0/launch/upcoming/?limit=3&mode=list
```

De cada lanzamiento extraigo el nombre, el cohete, la fecha y el sitio.

2.3.5 Open-Meteo

Pronóstico horario de nubosidad. Gratuito y sin clave.

```
https://api.open-meteo.com/v1/forecast?latitude={lat}&longitude={lon}&hourly=cloud_cover&timezone=auto&forecast_days=1
```

De la respuesta me quedo solo con las horas nocturnas (de 21:00 a 03:00), saco la media y la categorizo en despejado, parcialmente nublado o cubierto. Si la cobertura nocturna es menor del 40% pongo además una recomendación: «cielo favorable para observación astronómica».

2.3.6 Sunrise-Sunset.org

Devuelve las horas de salida y puesta del sol y, lo más interesante para mí, las del crepúsculo astronómico:

```
https://api.sunrise-sunset.org/json?lat={lat}&lng={lon}&formatted=0
```

2.3.7 arXiv astro-ph (RSS)

Feed RSS oficial de papers de astrofísica de arXiv. Me quedo con los seis primeros.

```
https://rss.arxiv.org/rss/astro-ph
```

2.3.8 SpaceNews (RSS)

Feed RSS de noticias del sector espacial. Cojo cinco titulares.

```
https://spacenews.com/feed/
```

2.4 Rama A — Briefing diario nodo a nodo

2.4.1 Nodo 1 — Schedule 08:00 diario

Trigger temporal con expresión cron `0 8 \* \* \*`. Dispara el resto de la rama todos los días a las 8 de la mañana.

2.4.2 Nodo 2 — Configuración (Code)

Devuelve un JSON con la configuración del día.

```
return [{ json: { lat: 37.608, lon: -0.987, ciudad: 'Cartagena, España',
tono: 'divulgativo', hoy: new Date().toISOString().slice(0, 10) } }];
```

`lat` y `lon` las consumen Open-Meteo y Sunrise-Sunset. `ciudad` aparece en el prompt y en el mensaje de Telegram. `hoy` es la fecha en formato ISO recortada a YYYY-MM-DD, que es lo que necesita NeoWs en su URL.

2.4.3 Nodos 3 a 10 — 8 nodos HTTP a las fuentes externas

Una serie de ocho nodos HTTP Request en cadena, cada uno consulta una fuente distinta (descritas en §2.3). Todos llevan `User-Agent` puesto a un valor explícito (algunas APIs como NASA APOD rechazan peticiones sin user-agent), un `timeout` de 30 segundos y, sobre todo, la opción `onError: continueRegularOutput`. Si una API se cae, el siguiente nodo se ejecuta igual y la consolidación se adapta.

2.4.4 Nodo 11 — Consolidar datos (Code)

Recibe la salida de los ocho HTTP previos y produce una estructura unificada. Es el primer nodo grande del workflow.

Lo primero es un helper para leer cada nodo de forma defensiva. Como las APIs llevan `onError: continueRegularOutput`, una de ellas puede haber fallado y la respuesta llegar vacía. Si simplemente accedo a un campo inexistente, el nodo Code peta. Por eso uso esto:

```
const getJson = (nodo) => { try { return $(nodo).first().json || {}; } catch(e) { return {}; } };
```

Después limpia el APOD, calcula la distancia ISS-Cartagena con haversine, extrae los asteroides, los lanzamientos, las nubes nocturnas y el crepúsculo. La parte del haversine es:

```
const toRad = d => d * Math.PI / 180; const dLat = toRad(iss.lat - ubic.lat);
const dLon = toRad(iss.lon - ubic.lon); const hav = Math.sin(dLat/2)**2
+ Math.cos(toRad(ubic.lat)) * Math.cos(toRad(iss.lat)) *
Math.sin(dLon/2)**2; iss.distancia_a_ti_km = Math.round( 6371 * 2 *
Math.atan2(Math.sqrt(hav), Math.sqrt(1-hav)) ); iss.sobre_zona =
iss.distancia_a_ti_km < 2000;
```

La fórmula del haversine es la estándar para calcular la distancia entre dos puntos sobre una esfera dadas sus latitudes y longitudes. 6371 es el radio medio de la Tierra en kilómetros.

Para los asteroides hago un `.slice(0, 3)` para quedarme solo con los tres más relevantes y de cada uno saco diámetro medio, distancia en distancias lunares y la bandera de potencialmente peligroso. Para los

lanzamientos paso por `slice(0, 3)` también y extraigo nombre, cohete, fecha y lugar.

Las nubes nocturnas las saco filtrando las horas entre 21 y 03 y haciendo la media. El crepúsculo astronómico lo formateo a HH:MM.

El parser RSS para arXiv y SpaceNews es un regex sencillo que extrae solo el `` de cada `<item>`:</p>
</div>
<div data-bbox="115 275 876 372" data-label="Text">
<pre>function parseRss(xml, maxItems) { if (!xml || typeof xml !== 'string') return
[]; const items = []; const re = /<item[>]*>([\s\S]*?)<\/item>/g; let m, c
= 0; while ((m = re.exec(xml)) && c < maxItems) { const tit =
(m[1].match(/<title[>]*>(?!<CDATA\[\]?([\s\S]*?)(?:\]\>)?<\/title>/) ||
[, ''])[1].trim(); if (tit) items.push({ titulo: tit }); c++; } return
items; }</pre>
</div>
<div data-bbox="115 383 886 428" data-label="Text">
<p>Al final del nodo se construye una lista plana de titulares prefijados que es lo que se mandará al embedding:</p>
</div>
<div data-bbox="115 447 881 511" data-label="Text">
<pre>const titulos = [...arxiv.map(i => 'PAPER: ' + i.titulo), ...space.map(i =>
'NOTICIA: ' + i.titulo), ...lanzamientos.map(l => 'LANZAMIENTO: ' + l.nombre + '
(' + l.cohete + ')'), ...asteroides.map(a => 'ASTEROIDE: ' + a.nombre + ' a ' +
a.distancia_lunar + ' distancias lunares')];</pre>
</div>
<div data-bbox="115 526 539 546" data-label="Section-Header">
<h3>2.4.5 Nodo 12 — Embeddings Ollama</h3>
</div>
<div data-bbox="115 552 627 570" data-label="Text">
<p>HTTP Request a la API de embeddings de Ollama:</p>
</div>
<div data-bbox="115 589 843 622" data-label="Text">
<pre>POST http://ollama:11434/api/embed { "model": "nomic-embed-text", "input":
["PAPER: ...", "NOTICIA: ...", ...] }</pre>
</div>
<div data-bbox="115 632 872 727" data-label="Text">
<p>La respuesta tiene un campo `embeddings` con un array de vectores, uno por cada string de entrada. Cada vector es un array de 768 números reales. Este nodo NO usa credencial de n8n. Llama directo por HTTP a la red interna de Docker.</p>
</div>
<div data-bbox="115 750 624 769" data-label="Section-Header">
<h3>2.4.6 Nodo 13 — Clustering y prompt (Code)</h3>
</div>
<div data-bbox="115 773 776 844" data-label="Text">
<p>Segundo nodo Code grande. Hace dos cosas: el clustering y la construcción del prompt. La similitud coseno está implementada manualmente:</p>
</div>
<div data-bbox="432 938 563 956" data-label="Page-Footer">Página 14 de 37</div>

```
function cos(a, b) { let dot = 0, na = 0, nb = 0; for (let i = 0; i < a.length; i++) { dot += a[i] * b[i]; na += a[i] * a[i]; nb += b[i] * b[i]; } return dot / (Math.sqrt(na) * Math.sqrt(nb)); }
```

El clustering greedy: voy uno a uno por los titulares; para cada uno calculo la similitud con el centroide de cada cluster que ya tenga; si supera 0,70, lo añado a ese cluster y recalculo el centroide; si no, abro cluster nuevo:

```
const UMBRAL = 0.70; const clusters = []; for (let i = 0; i < titulos.length; i++) { let asignado = false; for (const c of clusters) { if (cos(embs[i], c.centroide) > UMBRAL) { c.items.push(titulos[i]); for (let j = 0; j < c.centroide.length; j++) { c.centroide[j] = (c.centroide[j] * (c.items.length - 1) + embs[i][j]) / c.items.length; } asignado = true; break; } } if (!asignado) clusters.push({ centroide: [...embs[i]], items: [titulos[i]] }); }
```

El umbral 0,70 lo elegí probando: con 0,60 metía cosas no tan parecidas en el mismo cluster, y con 0,80 casi ningún titular se agrupaba. Con 0,70 se forman entre 4 y 8 grupos típicos, que es lo que tiene sentido para 16-20 titulares.

Después se construye el prompt para el LLM con reglas estrictas y los datos reales del día:

```
Eres un asistente de ciencia y espacio. Hablas en castellano a un usuario en {ciudad}. REGLAS ERICTAS: - Usa SOLO los datos que te paso abajo. NO inventes nada. - La ISS es un satélite, no un observatorio en tierra. - No uses emojis ni asteriscos ni markdown. Texto plano. - Máximo 9 frases. - Cubre: imagen del día, ISS, asteroides si los hay, lanzamientos si los hay, condiciones del cielo, 1-2 noticias o papers destacados.
```

Después del bloque de reglas viene el bloque de datos: APOD con título y descripción, ISS con coordenadas y distancia, asteroides cercanos, próximos lanzamientos, condiciones del cielo, y los grupos temáticos detectados por el clustering.

2.4.7 Nodo 14 — LLM Ollama (qwen2.5:1.5b)

HTTP Request a la API de generación de Ollama:

```
POST http://ollama:11434/api/generate { "model": "qwen2.5:1.5b", "prompt": "<prompt completo>", "stream": false, "keep_alive": "30m", "options": {
```

```
"temperature": 0.5,      "num_predict": 500,      "num_ctx": 3072,      "num_gpu": 0
} }
```

- ``stream: false`` para que la respuesta llegue de golpe en lugar de en chunks.
- ``keep_alive: 30m`` para que el modelo se quede cargado en RAM durante media hora. Sin esto, Ollama lo descarga después de 5 minutos y la siguiente ejecución tarda 10 segundos extra.
- ``temperature: 0.5`` — equilibrio entre coherencia y variedad.
- ``num_predict: 500`` — máximo 500 tokens en la respuesta.
- ``num_ctx: 3072`` — contexto de 3072 tokens.
- ``num_gpu: 0`` — fuerza CPU porque mi GPU GTX 960M ya no la soporta Ollama.

2.4.8 Nodo 15 — Telegram (briefing)

HTTP POST a la API de Telegram con el texto del briefing. Le quito asteriscos por si el modelo se ha pasado con markdown, y al final pego la URL del panel HTML del Apps Script. Telegram detecta la URL del APOD y la incrusta como imagen automáticamente.

2.4.9 Nodo 16 — Google Sheets (log)

Otro HTTP POST, esta vez al endpoint del Apps Script. Manda un JSON con timestamp, fecha, ciudad, datos del APOD, papers, noticias, bandera de visibilidad de la ISS, briefing completo y tokens consumidos.

2.5 Rama B — Chatbot nodo a nodo

2.5.1 Nodo 17 — Telegram Trigger (chatbot)

Configurado para escuchar updates de tipo ``message``. Cuando se activa el workflow, n8n registra automáticamente el webhook con Telegram a través de la URL pública de ngrok.

2.5.2 AI Agent y sub-nodos

El nodo central de la rama es el AI Agent (``@n8n/n8n-nodes-langchain.agent``). Acepta el mensaje del usuario, decide qué hacer

(responder directo o invocar herramientas) y produce la respuesta final.

Tiene cuatro sub-nodos enchufados:

- Modelo: Groq Chat Model con `llama-3.3-70b-versatile`.
- Memoria: Window Buffer de 6 turnos con `chat.id` como sessionKey, así cada usuario tiene su propio hilo aislado.
- Tool `ultimos\_briefings(n)` : HTTP GET al Apps Script con `?action=last&n={n}`.
- Tool `buscar\_briefing(q)` : HTTP GET con `?action=search&q={q}`.

El system message del agente le marca el comportamiento:

```
Eres COSMIC, un asistente conversacional experto en astronomía, espacio y ciencia.
Hablas en castellano con tono cercano pero riguroso. Ayudas al usuario a consultar
el histórico de Cosmic Briefings. REGLAS: - Si el usuario pregunta por contenido
del briefing, USA las herramientas. NO inventes contenido. - Si te saludan o
hacen small talk, responde sin llamar herramientas. - Cuando devuelvas resultados
de herramientas, RESÚMELOS de forma natural, no pegues JSON crudo. - Máximo 6
frases por respuesta. Sin markdown, sin asteriscos. - Si no sabes algo, dilo. No
alucines.
```

2.5.3 Nodo 18 — Telegram (respuesta chatbot)

HTTP POST que manda la respuesta del agente al chat de Telegram, con la misma limpieza de asteriscos que la rama del briefing.

2.6 Rama C — Manejo de errores

La rama C es un Error Trigger global y un nodo HTTP a Telegram. Si cualquier nodo del workflow falla y no se gestiona localmente (por ejemplo, un error de programación en un Code, o que la API de Groq devuelva 500), el Error Trigger se activa y manda un mensaje al chat del administrador con el texto del error y el nombre del nodo que falló. Esta rama es complementaria a los `onError: continueRegularOutput` de los nodos HTTP de la rama A.

2.7 Apps Script (Code.gs)

El Apps Script `Code.gs` es la capa que comunica n8n con Google Sheets. Vive como Web App pública desplegada con `Deploy → New deployment` desde el editor de Apps Script.

2.7.1 doPost: cómo se guarda un briefing

Recibe el JSON del nodo 16 y añade una fila a la hoja activa. Aplica una limpieza importante: si n8n ha enviado el JSON con un `=` al principio (un comportamiento conocido de las expresiones de n8n), lo elimina, y la función `c()` quita ese mismo `=` de cada valor individual antes de escribirlo. Sin esto, Sheets interpreta el contenido como una fórmula y muestra `#NAME?` en lugar del valor real.

```
function doPost(e) {
  var raw = e.postData.contents;
  if (raw.charAt(0) === '=')
    raw = raw.substring(1);
  var datos = JSON.parse(raw);
  var c = function(v) {
    if (v === null || v === undefined || v === 'null') return 'N/A';
    var s = String(v);
    while (s.charAt(0) === '=') s = s.substring(1);
    return s;
  };
  var hoja = SpreadsheetApp.getActiveSpreadsheet().getActiveSheet();
  hoja.appendRow([
    c(datos.timestamp), c(datos.fecha), c(datos.ciudad),
    c(datos.apod_titulo), c(datos.apod_url), c(datos.papers), c(datos.noticias),
    c(datos.iss_visible), c(datos.briefing_llm), c(datos.tokens)
  ]);
  ...
}
```

2.7.2 doGet: tres acciones en un solo endpoint

Lee el parámetro `action` y bifurca:

- `?action=last&n=N`: devuelve las últimas N filas en formato JSON.
- `?action=search&q=X`: filtra las filas en las que la cadena `X` aparezca (case-insensitive) en cualquier campo de texto relevante (briefing, noticias, papers o título APOD).
- `?action=dashboard`: invoca la función `renderDashboard`, que construye una página HTML con tarjetas, una por briefing.

2.7.3 renderDashboard: el panel HTML

Cuando llega `?action=dashboard`, en lugar de devolver JSON se devuelve una página HTML completa con tarjetas. Cada tarjeta tiene cabecera con título APOD y metadatos, la imagen del APOD enlazada a su URL original, y el texto del briefing con saltos de línea convertidos a

`<br>`. La etiqueta amarilla «ISS sobre tu zona» se muestra solo si en esa fila el campo `ISS\_Visible` es `SI`.

2.8 Decisiones técnicas y trazabilidad de ideas

2.8.1 Origen del proyecto

De dónde viene cada pieza importante del proyecto:

- Idea base. La sección 5 del enunciado de la tarea propone como ejemplo un «resumidor de noticias» con cron + RSS + LLM + Telegram. Esa fue mi punto de partida. Lo cambié de tema (de noticias generales a astronomía y espacio) y lo amplié con más fuentes, embeddings, clustering, panel HTML y un chatbot.
- Ejemplo de RAG visto en clase. El RAG sobre Google Drive que vimos no encajaba con lo que pedía el enunciado para nota máxima, así que lo descarté como núcleo. Pero me sirvió para entender cómo se enchufan embeddings, memoria y herramientas a un agente, y eso lo aproveché en la rama B.
- Self-hosted-ai-starter-kit. Kit oficial de n8n que reúne en un docker-compose los servicios típicos para experimentar con IA local. Lo bajé de <https://github.com/n8n-io/self-hosted-ai-starter-kit>.
- Modelos de Ollama. qwen2.5:1.5b lo elegí del catálogo público de Ollama (<https://ollama.com/library>) tras probar varios para el español. nomic-embed-text es el modelo de embeddings recomendado en la documentación oficial.
- Groq. La API key se saca en <https://console.groq.com/keys>. El modelo llama-3.3-70b-versatile lo elegí porque su tier gratuito permite tool calling sin restricciones.
- Fórmula del haversine. La estándar para calcular distancia entre dos puntos sobre la esfera terrestre. La implementé yo en JavaScript a partir de la fórmula matemática, no de un paquete.
- Clustering greedy. Algoritmo simple que se me ocurrió tras ver que un k-means o un HDBSCAN eran excesivos para 16-20 titulares.

Resumen: la chispa es del enunciado, la estructura del agente bebe del ejemplo de clase, el resto sale de mí o de la documentación oficial.

2.8.2 Local en lugar del n8n del profesor

La práctica daba acceso a una instancia compartida de n8n en ``progian8n.duckdns.org``. En la fase final del proyecto ese servidor estuvo varias horas inaccesible. Para no depender de un servicio fuera de mi control con la fecha de entrega encima, monté toda la infraestructura en mi máquina con el ``self-hosted-ai-starter-kit``. La sección 4 del enunciado contempla esta configuración para usuarios que quieran trabajar en local.

2.8.3 Groq para el chatbot, Ollama para el briefing

La GPU disponible es una NVIDIA GTX 960M con 2 GB de VRAM y compute capability 5.0. Las versiones actuales de Ollama ya no soportan esa arquitectura, así que toda la inferencia ocurre en CPU. Eso afecta a las dos ramas de forma distinta: el briefing diario tarda 5-10 segundos en CPU con ``qwen2.5:1.5b`` (asumible), pero el chatbot necesita tool calling fiable y rapidez.

Probé ``qwen2.5:3b`` en CPU y tardaba 30-60 segundos por respuesta, rompiendo el webhook de Telegram. Por eso muevo el chatbot a Groq, que es API gratuita y responde en milisegundos. La pieza que más se ejecuta (el briefing) se mantiene 100% local; solo lo interactivo va por la nube.

2.8.4 Un solo workflow

La instrucción de entrega del enunciado pide un único fichero llamado ``apellido_nombre_workflow.json``. Las tres ramas caben perfectamente en el mismo fichero compartiendo bot, hoja y modelo de embeddings.

2.8.5 onError: continueRegularOutput en las 8 fuentes

Si el workflow se rompe cuando una API falla, el briefing diario se queda sin entregar. Para evitarlo, los ocho nodos HTTP llevan activado ``onError``:

`continueRegularOutput``. La consolidación se ha programado de forma defensiva (con `try/catch` en cada bloque) y pone valores por defecto cuando no hay datos. El briefing siempre llega, aunque sea con menos información.

3. Instrucciones de configuración

3.1 Requisitos previos

- Docker Desktop instalado y arrancado.
- ngrok instalado, autenticado con la cuenta personal y con un dominio estático del plan gratuito (``https://ngrok.com/download``).
- El kit ``self-hosted-ai-starter-kit`` clonado en local (``https://github.com/n8n-io/self-hosted-ai-starter-kit``).

3.2 Arranque del kit en Docker

El kit trae 4 contenedores: ``n8n``, ``ollama``, ``postgres-1`` y ``qdrant``. Solo uso los tres primeros (no hago RAG, así que Qdrant sobra). Como el kit usa nombres internos distintos para los servicios del compose, lo más práctico es arrancar todo y parar Qdrant a mano:

```
cd C:\self-hosted-ai-starter-kit-main docker compose up -d docker stop qdrant
```

Compruebo que quedan los 3 en marcha:

```
docker ps
```

n8n queda accesible en ``http://localhost:5678``. La primera vez crea usuario y contraseña, las siguientes login normal.

3.3 Descarga de modelos en Ollama

El kit ya trae un contenedor llamado ``ollama`` con el binario instalado. En estos comandos, el primer ``ollama`` es el nombre del contenedor, el segundo es el binario:

```
docker exec -it ollama ollama pull qwen2.5:1.5b docker exec -it ollama ollama pull nomic-embed-text
```

Para comprobar que están:

```
Invoke-RestMethod -Uri "http://localhost:11434/api/tags" | ConvertTo-Json -Depth 4
```

3.4 Túnel ngrok

El chatbot recibe mensajes de Telegram y para eso n8n tiene que estar accesible desde fuera. Abro una PowerShell aparte y la dejo abierta mientras uso el chatbot:

```
ngrok http 5678 --domain=tapping-widget-entering.ngrok-free.dev
```

Si cierro esa ventana, el chatbot deja de funcionar. El briefing diario y el panel HTML van por su cuenta, no dependen de ngrok.

Para verificar que el túnel está vivo:

```
Invoke-RestMethod -Uri "http://127.0.0.1:4040/api/tunnels" | ConvertTo-Json -Depth 5
```

3.5 Credenciales en n8n

Solo hacen falta dos credenciales. En `http://localhost:5678` → menú lateral → `Credentials` → `New`:

- **Telegram API**: token del bot que me dio BotFather.
- **Groq**: API key gratis sacada en `https://console.groq.com/keys`. Importante copiarla en cuanto la genera, después solo se ve el prefijo.

Pulso **Test** en las dos. Si pasan, sigo. Los nodos del briefing diario (`Embeddings Ollama` y `LLM Ollama`) NO necesitan credencial: llaman a Ollama por HTTP directo a `http://ollama:11434` (la URL interna de Docker).

3.6 Despliegue del Apps Script

Abro la hoja Cosmic Briefing en Google Sheets → `Extensiones` → `Apps Script`. Pego el contenido de `google-apps-script/Code.gs` en el editor y le doy a:

```
Deploy → Manage deployments → (lápiz para editar) → Version: New version → Deploy
```

La URL `/exec` no cambia entre versiones. Para comprobar que el panel HTML responde, la pego en el navegador con `?action=dashboard` al final.

3.7 Importación y activación del workflow

En n8n → arriba a la izquierda → `+` → `Import from File` → selecciono `ferrer\_antonio\_workflow.json`.

Asigno credenciales en los dos nodos que las piden:

- `17. Telegram Trigger (chatbot)` → Telegram API.
- `Groq Chat Model (llama-3.3-70b)` → Groq.

El resto de nodos HTTP no necesitan credencial, llaman a APIs públicas. Activo el workflow con el toggle de arriba a la derecha. Al activarse, n8n registra solo el webhook con Telegram a través de la URL de ngrok.

4. Capturas de pantalla

4.1 Vistas del workflow

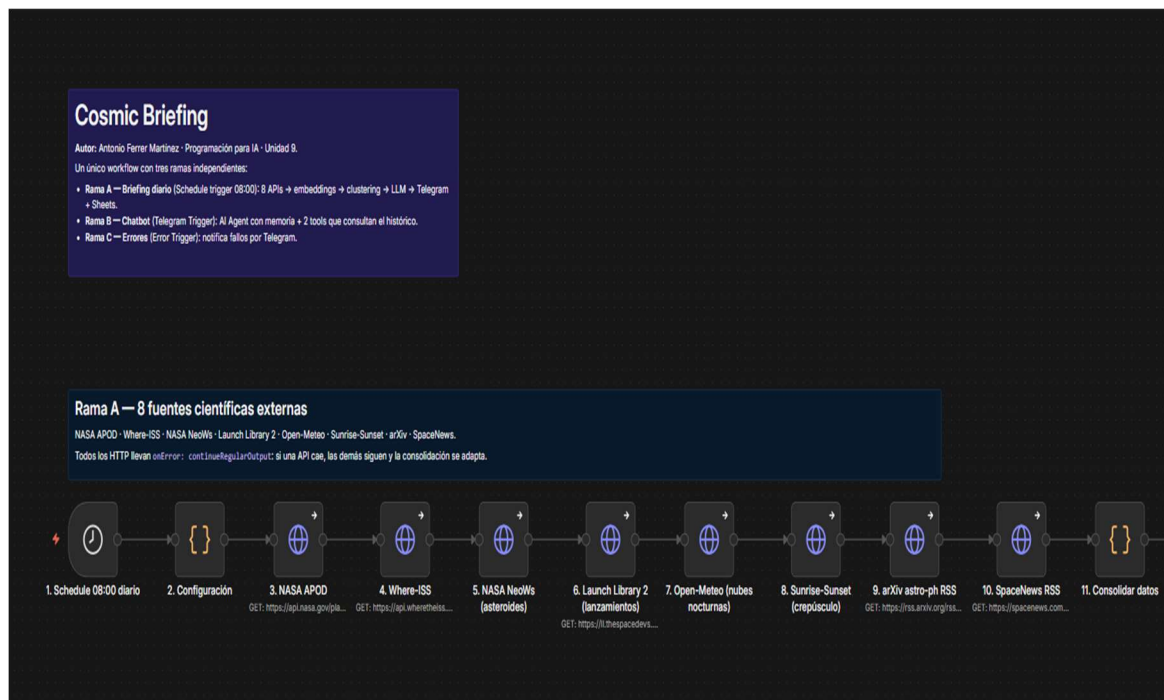


Figura 1.A. Workflow — Rama A (parte izquierda). Cabecera, sticky de las 8 fuentes y nodos 1-11 (Schedule, Configuración, las 8 APIs y Consolidar datos).

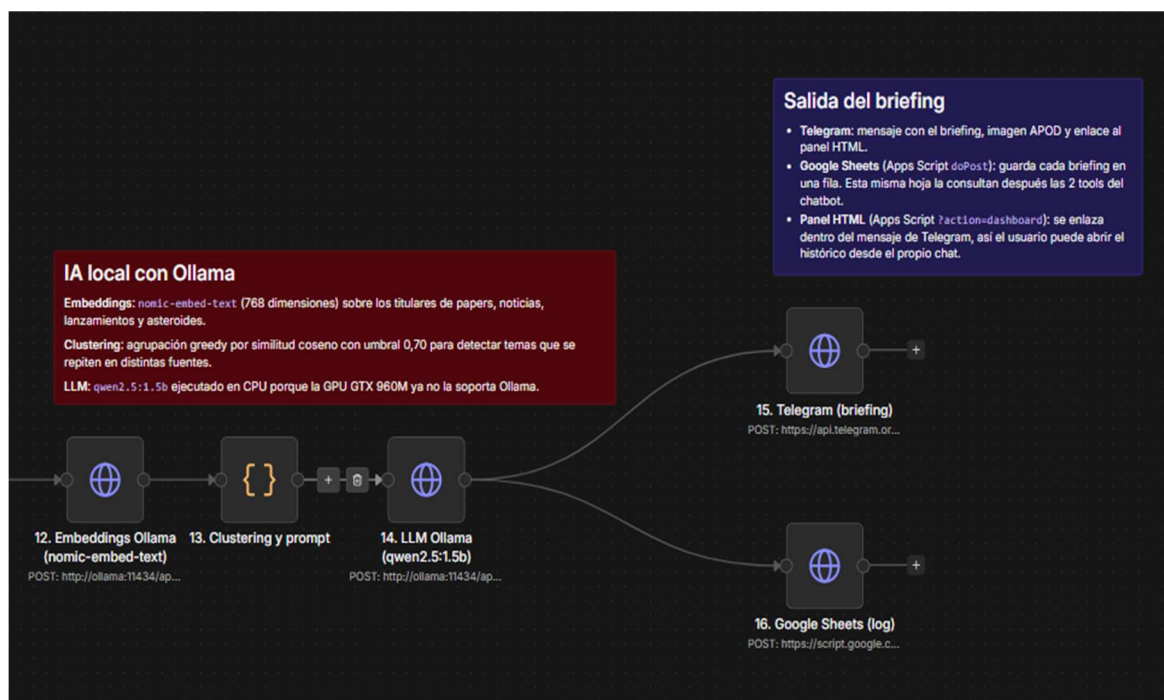


Figura 1.B. Workflow — Rama A (parte derecha). Sticky de IA local con Ollama, sticky de la salida del briefing y los nodos 12-16 (Embeddings, Clustering, LLM, Telegram briefing y Sheets).

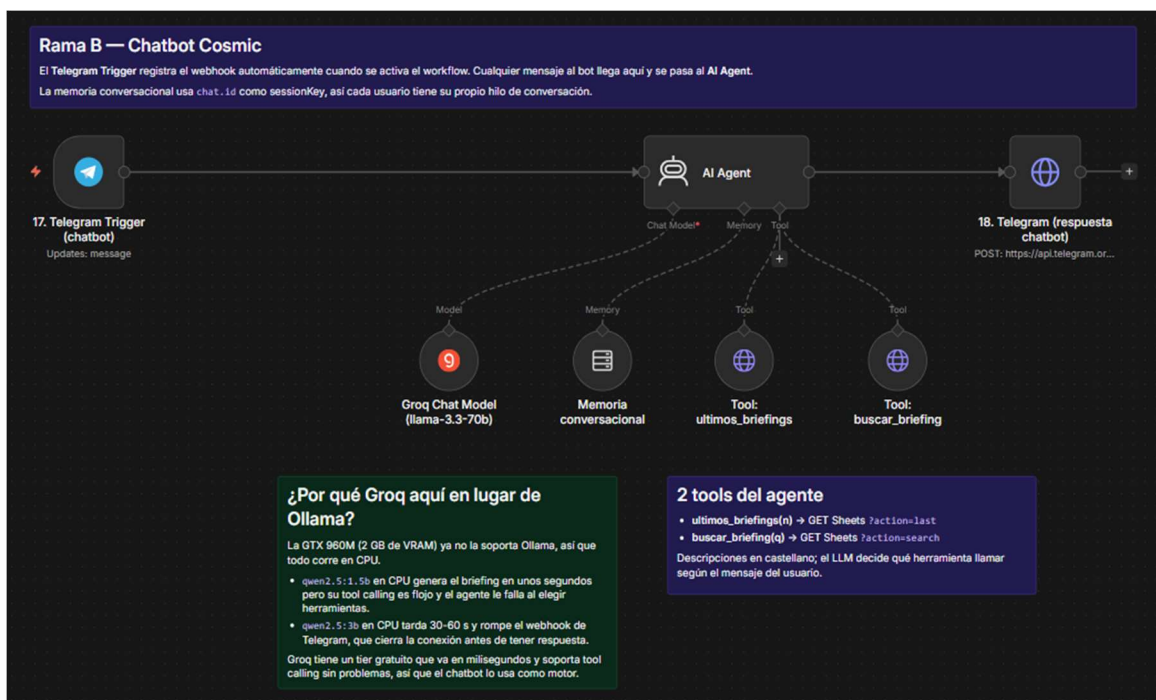


Figura 2. Workflow — Rama B (chatbot). Telegram Trigger → AI Agent → Telegram (respuesta), con los cuatro sub-nodos enchufados al agente: Groq Chat Model, memoria conversacional, tool ultimos_briefings y tool buscar_briefing.

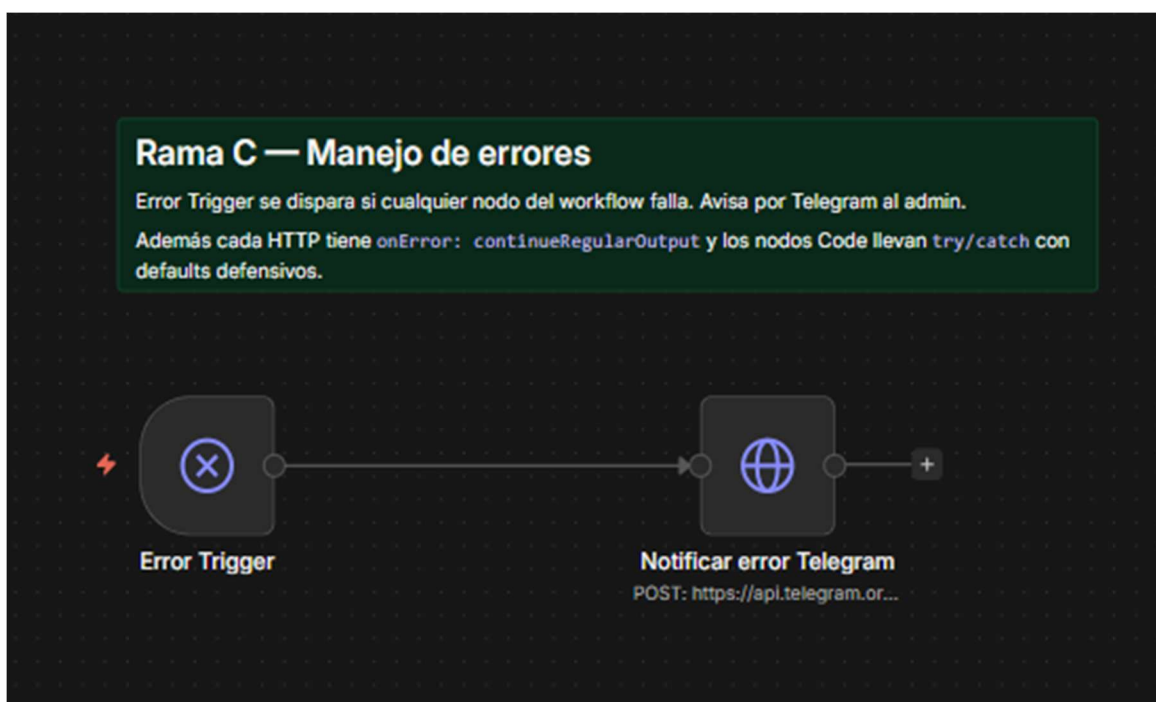


Figura 3. Workflow — Rama C (errores). Error Trigger global y el nodo que avisa por Telegram al admin.

4.2 Ejecución exitosa

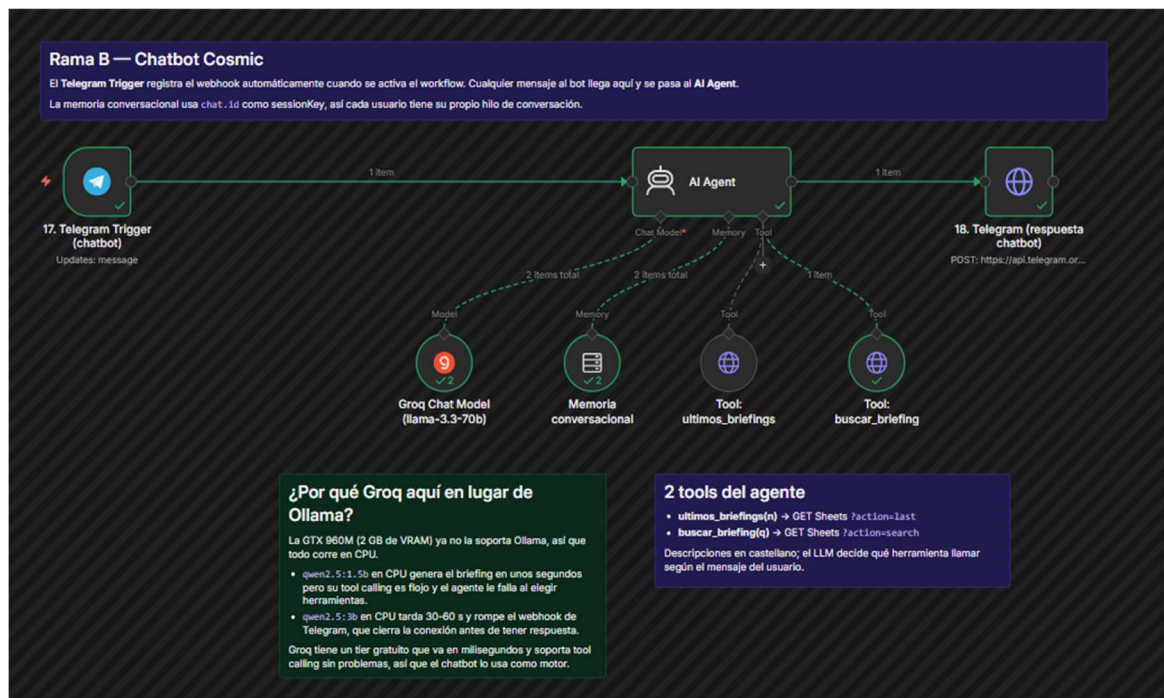


Figura 4. Ejecución del chatbot funcionando. Pregunta «Busca algo sobre SpaceX»: todos los nodos en verde, Groq se llama dos veces (decidir tool y redactar) y buscar_briefing lanza una petición al Apps Script.

4.3 Mensajes en Telegram

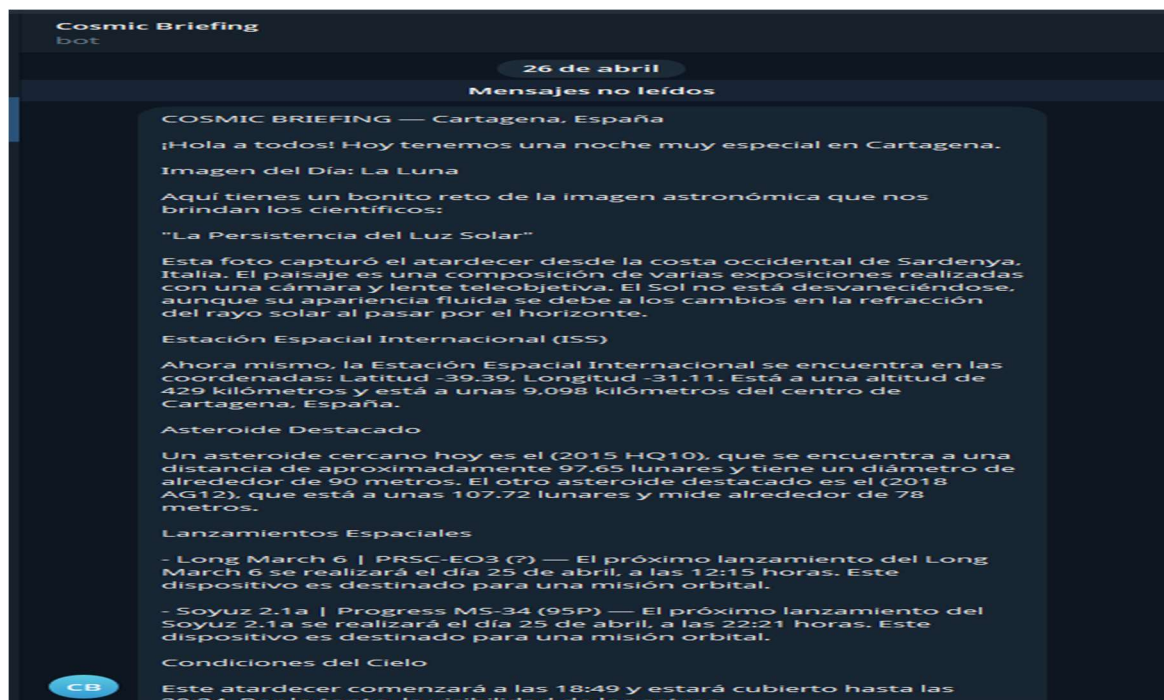


Figura 5.A. Briefing diario en Telegram (parte superior). Cabecera con la ciudad, saludo, imagen astronómica del día (APOD), posición de la ISS, asteroides cercanos detectados

(NeoWs), próximos lanzamientos (Launch Library 2) y comienzo de las condiciones del cielo.

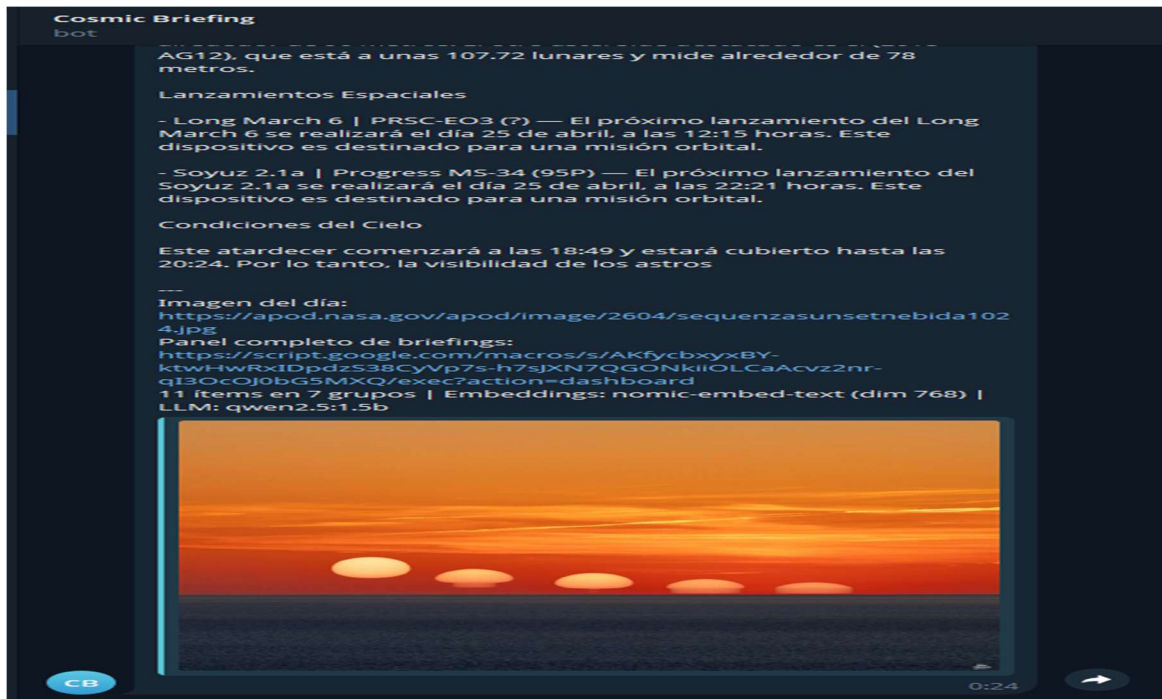


Figura 5.B. Briefing diario en Telegram (parte inferior). Continuación con las condiciones de observación nocturnas, pie con la URL del APOD, enlace al panel HTML y la línea de métricas. Telegram incrusta automáticamente la imagen del APOD.

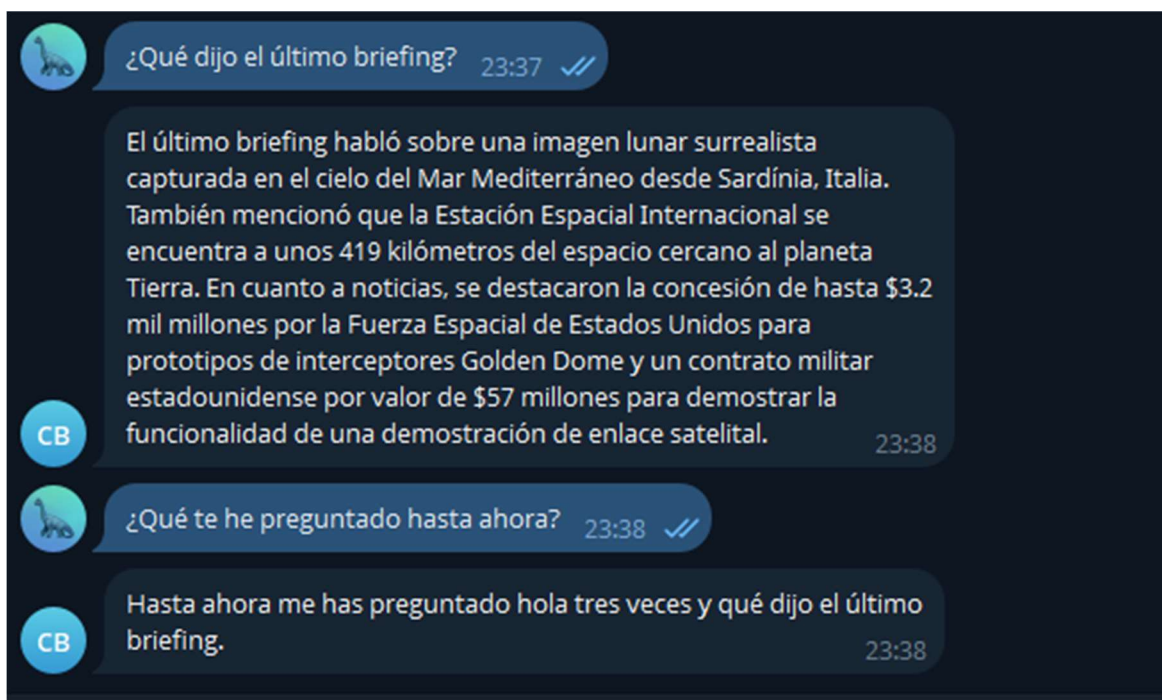


Figura 6. Conversación con el chatbot. El bot recibe «¿Qué dijo el último briefing?» y resume el contenido (ha llamado a `ultimos_briefings`). Después se le pregunta qué se le ha preguntado antes y lo recuerda, lo que confirma la memoria.

4.4 Persistencia y panel

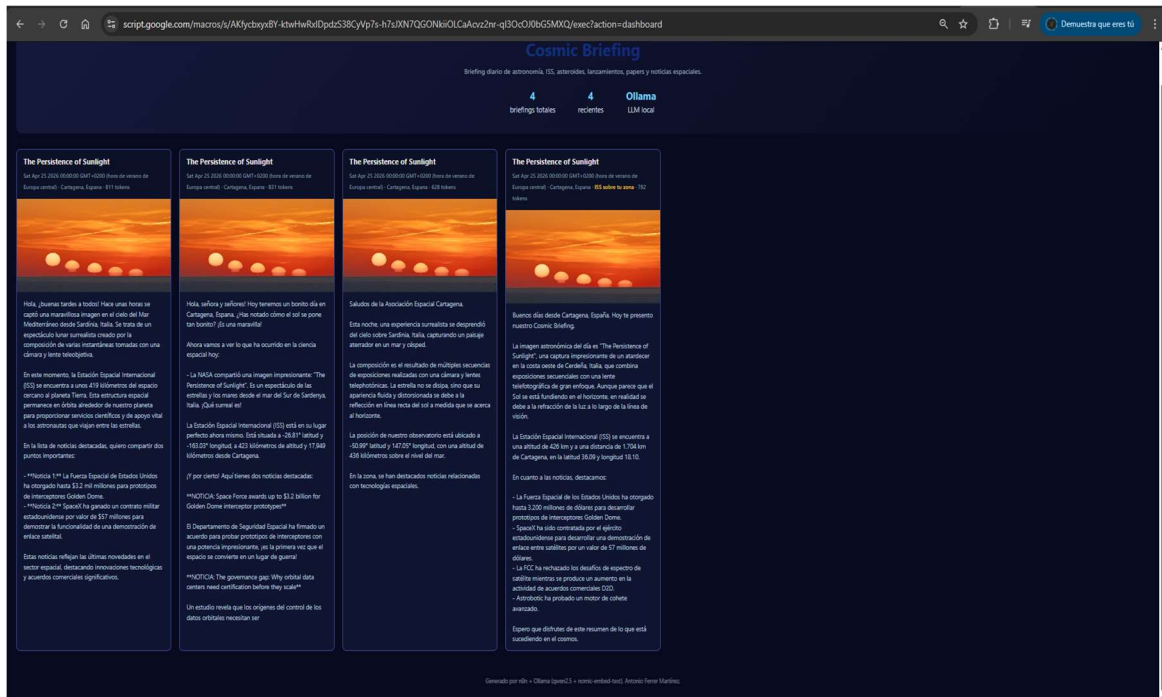


Figura 7. Panel HTML del histórico. Salida del endpoint `?action=dashboard` del Apps Script: cabecera con los contadores y, debajo, una tarjeta por briefing.

	A	B	C	D	E	F	G	H	I	J	K	L	M	N
1	Timestamp	Fecha	Ciudad	APOD_Titulo	APOD_URL	Papers	Noticias	ISS_Visible	Briefing_LLM	Tokens				
2									Buenos días desde Cartagena, España. Hoy te presento nuestro Cosmic Briefing.					
									La imagen astronómica del día es "The Persistence of Sunlight", una captura impresionante de la					
									La Estación Espacial Internacional (ISS) se encuentra a una altitud de 426 km y a una distanc					
									En cuanto a las noticias, destacamos:					
									- La Fuerza Espacial de los Estados Unidos ha otorgado hasta 3.200 millones de dólares para					
									- SpaceX ha sido contratada por el ejército estadounidense para desarrollar una demostración					
									- La FCC ha rechazado los desafíos de espectro de satélite mientras se produce un aumento e					
									- Astroblet ha probado un motor de cohete avanzado.					
	2026-04-25T07:	2026-04-25	Cartagena, Esp	The Persistence	https://apod.nasa.gov/apod/imag	Space Force aw SI			Espero que disfrutes de este resumen de lo que está sucediendo en el cosmos.	782				
									Saludos de la Asociación Espacial Cartagena.					
									Esta noche, una experiencia surrealista se desprendió del cielo sobre Sardinia, Italia, capturan					
									La composición es el resultado de múltiples secuencias de exposiciones realizadas con una cá					
									La posición de nuestro observatorio está ubicado a -50.99° latitud y 147.05° longitud, con una i					
									En la zona, se han destacado noticias relacionadas con tecnologías espaciales.					
	2026-04-25T08:	2026-04-25	Cartagena, Esp	The Persistence	https://apod.nasa.gov/apod/imag	Space Force aw NO			Hola, señora y señores! Hoy tenemos un bonito día en Cartagena, España. ¿Has notado cómo	628				
									Ahora vamos a ver lo que ha ocurrido en la ciencia espacial hoy:					
									- La NASA compartió una imagen impresionante: "The Persistence of Sunlight". Es un especia					
									La Estación Espacial Internacional (ISS) está en su lugar perfecto ahora mismo. Está situada a					
									¡Y por cierto! Aquí tienes dos noticias destacadas:					
									"NOTICIA: Space Force awards up to \$3.2 billion for Golden Dome Interceptor prototypes"					
									El Departamento de Seguridad Espacial ha firmado un acuerdo para probar prototipos de interi					
									"NOTICIA: The governance gap: Why orbital data centers need certification before they scale"					
	2026-04-25T08:	2026-04-25	Cartagena, Esp	The Persistence	https://apod.nasa.gov/apod/imag	Space Force aw NO			Un estudio revela que los orígenes del control de los datos orbitales necesitan ser	831				
									Hola, ¿buenas tardes a todos! Hace unas horas se captó una maravillosa imagen en el cielo de					
									En este momento, la Estación Espacial Internacional (ISS) se encuentra a unos 419 kilómetros					

Figura 8. Hoja de Google Sheets con las cabeceras correctas y los briefings persistidos. Esta misma hoja la consultan después las dos tools del chatbot.

4.5 ngrok funcionando

```

ngrok

♦ One gateway for every AI model. Available in early access *now*: https://ngrok.com/r/ai

Session Status      online
Account             antonio.ferrer@gmail.com (Plan: Free)
Update              update available (version 3.38.0, Ctrl-U to update)
Version              3.36.1-msix-stable
Region              Europe (eu)
Latency              821ms
Web Interface        http://127.0.0.1:4040
Forwarding            https://tapping-widget-entering.ngrok-free.dev -> http://localhost:5678

Connections
  ttl   opn   rt1   rt5   p50   p90
    6     0   0.00  0.00  6.20  6.36

HTTP Requests
-----
14:18:22.614 CEST POST /webhook/cosmic-chatbot/webhook 200 OK
14:16:38.099 CEST POST /webhook/cosmic-chatbot/webhook 200 OK
14:16:10.883 CEST POST /webhook/cosmic-chatbot/webhook 200 OK
13:34:48.473 CEST POST /webhook/cosmic-chatbot/webhook 200 OK
13:34:41.233 CEST POST /webhook/cosmic-chatbot/webhook 200 OK
13:31:24.178 CEST POST /webhook/cosmic-chatbot/webhook 200 OK

```

Figura 9. ngrok exponiendo n8n al exterior. Sesión activa con el dominio estático apuntando a localhost:5678. El log de HTTP Requests muestra los POST de Telegram entregándose con 200 OK.

5. Problemas encontrados y cómo los resolví

Lista de los problemas reales que aparecieron durante el desarrollo, con la solución aplicada en cada caso.

Problema	Solución
Las celdas de Sheets se llenaban con #NAME?	n8n insertaba un = al inicio de los valores. Se elimina en el Apps Script y en la consolidación.
Los \n aparecían como literal en Telegram	En las expresiones de n8n hay que escapar dos veces: \\n.
Ollama crasheaba al usar GPU	La GTX 960M (compute 5.0) ya no la soporta. Forzado CPU con options.num_gpu: 0.
Modelos 3B no entraban en VRAM	Cambiado a qwen2.5:1.5b para el briefing y a Groq para el chatbot.
El modelo se descargaba de RAM entre ejecuciones	Activado keep_alive: 30m en el body de la petición.
El LLM ponía asteriscos pese al prompt	Post-procesado con regex en el nodo Telegram para quitarlos.
El briefing salía cortado a mitad de frase	Ampliado num_predict a 500 y num_ctx a 3072.
Tool calling con qwen pequeño no era fiable	Movido el chatbot a Groq con llama-3.3-70b-versatile.
Si una API caía, se rompía toda la cadena	Añadido onError: continueRegularOutput en los 8 nodos HTTP.
El n8n del profesor no respondía	Migrado a un n8n local en Docker expuesto con ngrok.

Problema	Solución
ngrok hay que arrancarlo a mano cada vez	Documentado en §3.4. Mejora pendiente: meterlo como contenedor del kit con NGROK_AUTHTOKEN en .env.
Cabeceras de Sheets desplazadas una columna	Faltaba la columna Timestamp al inicio. Corregidas las cabeceras de la fila 1.
Webhook de Telegram daba 404	La URL pública apuntaba a un path que no existía. Forzado el setWebhook con la URL correcta.
Webhook de Telegram daba 403	Falta el secret token de n8n. Solución: desactivar y reactivar el workflow para que n8n re-registre el webhook.

6. Pruebas funcionales

Todas las pruebas que cito a continuación están grabadas en el vídeo demostrativo del proyecto, disponible en <https://www.youtube.com/watch?v=JRhiNauBYrM>. Resumido por escrito, las pruebas son:

- Ejecución manual del briefing desde el editor de n8n. El workflow recorre los 16 nodos en cadena, todos terminan en verde, y un mensaje completo llega a Telegram con todos los bloques (APOD, ISS, asteroides, lanzamientos, cielo, papers y noticias).
- Verificación de la persistencia. Después de la ejecución manual, una nueva fila aparece en Sheets con todas las columnas correctamente rellenas.
- Verificación del panel HTML. El enlace «Panel completo de briefings» que viene en cada mensaje de Telegram abre la página con todas las tarjetas históricas.
- Saludo simple al chatbot («Hola»). El agente responde sin invocar herramientas. En Executions se ve que la rama B se disparó pero no hay tick verde en las tools.
- Pregunta por el último briefing («¿Qué dijo el último briefing?»). El agente invoca `ultimos\_briefings`, recibe la última fila y la resume.
- Búsqueda por palabra clave («Busca algo sobre SpaceX»). El agente invoca `buscar\_briefing` con `q=SpaceX`, recibe los hits y los resume.
- Verificación de la memoria conversacional («¿Qué te he preguntado hasta ahora?»). El agente recuerda los turnos previos del mismo `chat.id` y los enumera.
- Verificación del túnel ngrok. El panel local de ngrok en `127.0.0.1:4040` muestra el túnel activo y los logs de POST entregados a `/webhook/cosmic-chatbot/webhook` con código 200 OK.

7. Conclusiones y trabajos futuros

7.1 Conclusiones

El proyecto cumple con los objetivos planteados en §1.1. El briefing diario se entrega de forma autónoma sin intervención humana, integra ocho fuentes científicas reales, aplica embeddings y clustering para mejorar la coherencia narrativa, y produce un texto natural en castellano. El chatbot responde a consultas sobre el histórico mediante function calling y memoria conversacional, demostrando el uso del nodo AI Agent de n8n con varias herramientas.

A nivel personal el proyecto me ha permitido tocar de cerca tecnologías de IA generativa con una práctica real (no un ejemplo de juguete) y enfrentarme a problemas técnicos concretos: limitaciones de hardware, peculiaridades de las APIs externas, formatos de datos heterogéneos, gestión de webhooks, comportamiento de los LLM pequeños frente a function calling, etc. La separación entre Ollama local para tareas controladas (briefing) y Groq en la nube para la pieza interactiva (chatbot) ha sido la decisión más útil del proyecto.

La principal lección aprendida es que la robustez en pipelines con muchas dependencias externas se consigue con tolerancia a fallos a nivel de cada llamada (``onError``, ``try/catch``, defaults defensivos), no esperando a que todo vaya perfecto. Un solo ``onError: continueRegularOutput`` bien colocado evita una cantidad importante de incidentes.

7.2 Trabajos futuros

El proyecto se entrega funcional, pero quedan mejoras anotadas para una próxima iteración:

- Contenerizar ngrok dentro del propio docker-compose del kit, usando una variable NGROK_AUTHTOKEN en el .env, de forma que el túnel arranque junto al resto de servicios y desaparezca el paso manual.

- Sustituir la búsqueda por palabra clave del chatbot por una búsqueda semántica real, embedando también las preguntas y comparando con los embeddings de los briefings persistidos.
- Soportar varias ciudades simultáneamente, parametrizando la rama A para que un mismo workflow pueda generar briefings personalizados para distintos chats de Telegram.
- Añadir métricas de seguimiento (tiempo de cada nodo, número de hits a cada API, tokens consumidos por modelo) en una hoja aparte para detectar degradaciones.
- Sustituir el clustering greedy por uno aglomerativo o basado en HDBSCAN, que daría mejores agrupaciones cuando el número de titulares crezca.

8. Recursos y referencias

Lista de páginas que he tenido abiertas mientras montaba el proyecto.

8.1 Plataforma y entorno

- n8n self-hosted-ai-starter-kit — <https://github.com/n8n-io/self-hosted-ai-starter-kit>
- Documentación oficial de n8n — <https://docs.n8n.io/>
- AI Agent en n8n — <https://docs.n8n.io/integrations/builtin/cluster-nodes/root-nodes/n8n-nodes-langchain.agent/>
- Window Buffer Memory — <https://docs.n8n.io/integrations/builtin/cluster-nodes/sub-nodes/n8n-nodes-langchain.memorybufferwindow/>
- Tool HTTP Request — <https://docs.n8n.io/integrations/builtin/cluster-nodes/sub-nodes/n8n-nodes-langchain.toolhttprequest/>
- ngrok — <https://ngrok.com/download>

8.2 Modelos y LLM

- Ollama — <https://ollama.com>
- Catálogo de modelos de Ollama — <https://ollama.com/library>
- API de Ollama — <https://github.com/ollama/ollama/blob/main/docs/api.md>
- Groq — <https://console.groq.com/docs/quickstart>

8.3 APIs de datos del briefing

- NASA APOD y NeoWs — <https://api.nasa.gov/>
- Where the ISS at — <https://wheretheiss.at/w/developer>
- Launch Library 2 (TheSpaceDevs) — <https://ll.thespacedevs.com/2.2.0/swagger/>
- Open-Meteo — <https://open-meteo.com/en/docs>

- Sunrise-Sunset.org — <https://sunrise-sunset.org/api>
- arXiv RSS feeds — <https://info.arxiv.org/help/rss.html>
- SpaceNews RSS — <https://spacenews.com/feed/>

8.4 Telegram y Apps Script

- Telegram Bot API — <https://core.telegram.org/bots/api>
- Google Apps Script HtmlService — <https://developers.google.com/apps-script/reference/html/html-service>

8.5 Inspiración para el formato del proyecto

- Ejemplo «Resumidor de noticias» propuesto en la sección 5 del enunciado de la tarea (cron + RSS + LLM + Telegram), adaptado a astronomía y ampliado.
- Ejemplo de RAG visto en clase, que sirvió como referencia para entender cómo se enchufan embeddings, memoria y herramientas a un agente — aunque deliberadamente NO se ha realizado un RAG.